

Assignment No. - 2

ON

Full Stack Development – SSCA4020

TITLE: Student management system

BACHLEOR OF SCIENCE IN INFORMATION TECHNOLOGY
(HONOURS.)

SUBMITTED BY:

Name: Shah Rishabh Alpeshbhai

Enrolment: 23SS02IT181

Class: BSCIT-H – 7A - Batch 2023-27

INPUT:-

INDEX.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<meta name="description" content="Manage student records with search, filters, and statistics.">
```

```
<title>Student Management System</title>
```

```
<link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<header class="page-header">
```

```
<h1>Student Management System</h1>
```

```
<p class="subtitle">Add, search, filter, and manage student records</p>
```

```
</header>
```

```
<section class="panel" aria-labelledby="form-heading">
```

```
<h2 id="form-heading" class="section-title">Student Details</h2>
```

```
<form id="studentForm" novalidate>
```

```
<input type="hidden" id="editId" value="">
```

```

<div class="form-grid">
  <div class="field">
    <label for="studentId">Student ID</label>
    <input type="text" id="studentId" name="studentId" placeholder="e.g.
STU001" required
      autocomplete="off" pattern="[A-Za-z0-9_-]+" title="Letters, numbers,
hyphens, underscores">
  </div>

  <div class="field">
    <label for="name">Name</label>
    <input type="text" id="name" name="name" placeholder="Full name"
required autocomplete="name">
  </div>

  <div class="field">
    <label for="email">Email</label>
    <input type="email" id="email" name="email"
placeholder="email@example.com" required
      autocomplete="email">
  </div>

  <div class="field">
    <label for="mobile">Mobile</label>
    <input type="tel" id="mobile" name="mobile" placeholder="10-digit
number" required
      pattern="[0-9]{10}" title="Enter a 10-digit mobile number"
autocomplete="tel">
  </div>

```

```

<div class="field">

  <label for="course">Course</label>

  <input type="text" id="course" name="course" placeholder="e.g. Computer
Science" required>

</div>


<div class="field">

  <label for="age">Age</label>

  <input type="number" id="age" name="age" placeholder="0-100" required
min="5" max="100">

</div>


<div class="field">

  <label for="percentage">Percentage</label>

  <input type="number" id="percentage" name="percentage"
placeholder="0-100" required min="0"
max="100" step="0.01">

</div>

</div>


<div class="form-actions">

  <button type="submit" id="submitBtn" class="btn btn-primary">  Save
Student</button>

  <button type="button" id="deleteLastBtn" class="btn btn-delete-last"> 
Delete Last</button>

  <button type="button" id="cancelBtn" class="btn btn-secondary
hidden">Cancel Edit</button>

</div>

</form>

</section>

```

```

<section class="panel" aria-labelledby="controls-heading">
  <h2 id="controls-heading" class="section-title">Search & Filters</h2>

  <div class="search-bar">
    <label for="search" class="visually-hidden">Search students</label>
    <input type="search" id="search" placeholder="Search by name, ID, or course..."
autocomplete="off">
  </div>

  <div class="toolbar">
    <div class="toolbar-group" role="group" aria-label="Filters">
      <span class="toolbar-label">Filter</span>
      <button type="button" class="btn btn-filter" data-
filter="percentage">Percentage > 75</button>
      <button type="button" class="btn btn-filter" data-filter="age">Age >
20</button>
      <button type="button" class="btn btn-filter active" data-filter="all">Show
All</button>
    </div>

    <div class="toolbar-group" role="group" aria-label="Sort options">
      <span class="toolbar-label">Sort</span>
      <button type="button" class="btn btn-sort" data-
sort="name">Name</button>
      <button type="button" class="btn btn-sort" data-sort="age">Age</button>
      <button type="button" class="btn btn-sort" data-
sort="percentage">Percentage</button>
    </div>
  </div>
</section>

```

```

<section class="stats" aria-live="polite" aria-atomic="true">
  <div class="stat-card">
    <span class="stat-label">Total Students</span>
    <span class="stat-value" id="totalStudents">0</span>
  </div>
  <div class="stat-card">
    <span class="stat-label">Average Percentage</span>
    <span class="stat-value" id="avgPercentage">0</span>
  </div>
  <div class="stat-card">
    <span class="stat-label">Showing</span>
    <span class="stat-value" id="showingCount">0</span>
  </div>
</section>

<section class="panel table-panel" aria-labelledby="table-heading">
  <h2 id="table-heading" class="section-title">Student List</h2>
  <!-- Select All / Delete Selected Toolbar -->
  <div class="table-toolbar">
    <div class="table-toolbar-left">
      <input type="checkbox" id="selectAll" onchange="toggleAllCheckboxes()">
      <label for="selectAll">Select All</label>
    </div>
    <div class="table-toolbar-right">
      <button type="button" class="btn btn-delete-selected"
onclick="deleteSelected()" id="deleteSelectedBtn">
         Delete Selected (<span id="selectedCount">0</span>)
      </button>
    </div>
  </div>

```

```

</div>

<div class="table-wrapper">

  <table>

    <thead>

      <tr>

        <th scope="col" style="width: 40px;">

          <input type="checkbox" id="selectAllHeader"
onchange="toggleAllCheckboxes()">

        </th>

        <th scope="col">ID</th>

        <th scope="col">Name</th>

        <th scope="col">Email</th>

        <th scope="col">Mobile</th>

        <th scope="col">Course</th>

        <th scope="col">Age</th>

        <th scope="col">Percentage</th>

        <th scope="col">Actions</th>

      </tr>

    </thead>

    <tbody id="studentTableBody"></tbody>

  </table>

</div>

</section>

</div>

<div id="toast" class="toast hidden" role="status" aria-live="polite"></div>

<script src="script.js"></script>

</body>

</html>

```

Style.css

/* Student Management System */

```
:root {  
  --color-bg: #f0f4f8;  
  --color-surface: #ffffff;  
  --color-border: #e2e8f0;  
  --color-text: #1e293b;  
  --color-muted: #64748b;  
  --color-primary: #2563eb;  
  --color-primary-hover: #1d4ed8;  
  --color-secondary: #94a3b8;  
  --color-secondary-hover: #64748b;  
  --color-success: #16a34a;  
  --color-warning: #ea580c;  
  --color-danger: #dc2626;  
  --color-danger-hover: #b91c1c;  
  --radius: 10px;  
  --shadow: 0 4px 24px rgba(15, 23, 42, 0.08);  
  --font: "Segoe UI", system-ui, -apple-system, sans-serif;  
}  
  
*,  
*::before,  
*::after {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;
```



```
}
```

```
body {  
  font-family: var(--font);  
  background: var(--color-bg);  
  color: var(--color-text);  
  line-height: 1.5;  
  padding: 24px 16px 48px;  
  min-height: 100vh;  
}
```

```
.container {  
  width: min(1200px, 100%);  
  margin: 0 auto;  
}
```

```
.page-header {  
  text-align: center;  
  margin-bottom: 28px;  
}
```

```
.page-header h1 {  
  font-size: clamp(1.5rem, 4vw, 2rem);  
  font-weight: 700;  
  letter-spacing: -0.02em;  
}
```

```
.subtitle {  
  margin-top: 6px;
```

```
color: var(--color-muted);
font-size: 0.95rem;
}

.panel {
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  padding: 20px 24px;
  margin-bottom: 20px;
}

.section-title {
  font-size: 1rem;
  font-weight: 600;
  margin-bottom: 16px;
  color: var(--color-text);
}

.form-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  gap: 14px;
}

.field {
  display: flex;
  flex-direction: column;
```

```
gap: 6px;  
}
```

```
.field label {  
  font-size: 0.8rem;  
  font-weight: 600;  
  color: var(--color-muted);  
  text-transform: uppercase;  
  letter-spacing: 0.04em;  
}
```

```
input,  
button {  
  font-family: inherit;  
  font-size: 0.95rem;  
}
```

```
input {  
  padding: 10px 12px;  
  border: 1px solid var(--color-border);  
  border-radius: 8px;  
  background: var(--color-surface);  
  color: var(--color-text);  
  transition: border-color 0.2s, box-shadow 0.2s;  
}
```

```
input:disabled {  
  background: #f8fafc;  
  color: var(--color-muted);  
}
```

```
    cursor: not-allowed;
}

input:focus {
    outline: none;
    border-color: var(--color-primary);
    box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.15);
}

input:focus-visible,
button:focus-visible {
    outline: 2px solid var(--color-primary);
    outline-offset: 2px;
}

.form-actions {
    display: flex;
    flex-wrap: wrap;
    gap: 10px;
    margin-top: 18px;
}

.btn {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    padding: 10px 18px;
    border: none;
    border-radius: 8px;
```

```
font-weight: 600;
cursor: pointer;
transition: background 0.2s, transform 0.15s, opacity 0.2s;
}

.btn:active {
  transform: scale(0.98);
}

.btn-primary {
  background: var(--color-primary);
  color: #fff;
}

.btn-primary:hover {
  background: var(--color-primary-hover);
}

.btn-secondary {
  background: #e2e8f0;
  color: var(--color-text);
}

.btn-secondary:hover {
  background: #cbd5e1;
}

/* Delete Last Button */
.btn-delete-last {
```

```
background: #ef4444;  
color: #fff;  
border: none;  
padding: 10px 18px;  
border-radius: 8px;  
font-weight: 600;  
cursor: pointer;  
transition: background 0.2s, transform 0.15s;  
}
```

```
.btn-delete-last:hover {  
    background: #dc2626;  
}
```

```
.btn-delete-last:active {  
    transform: scale(0.98);  
}
```

```
.btn-delete-last:disabled {  
    opacity: 0.5;  
    cursor: not-allowed;  
}
```

```
.search-bar {  
    margin-bottom: 16px;  
}
```

```
#search {  
    width: 100%;
```

```
padding: 12px 14px;  
}
```

```
.toolbar {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;  
}
```

```
.toolbar-group {  
  display: flex;  
  flex-wrap: wrap;  
  align-items: center;  
  gap: 8px;  
}
```

```
.toolbar-label {  
  font-size: 0.75rem;  
  font-weight: 700;  
  text-transform: uppercase;  
  letter-spacing: 0.06em;  
  color: var(--color-muted);  
  margin-right: 4px;  
}
```

```
.btn-filter,  
.btn-sort {  
  background: #f1f5f9;  
  color: var(--color-text);
```

```
padding: 8px 14px;  
font-size: 0.85rem;  
border: 1px solid transparent;  
}
```

```
.btn-filter:hover,  
.btn-sort:hover {  
    background: #e2e8f0;  
}
```

```
.btn-filter.active,  
.btn-sort.active {  
    background: var(--color-primary);  
    color: #fff;  
    border-color: var(--color-primary);  
}
```

```
.stats {  
    display: grid;  
    grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));  
    gap: 14px;  
    margin-bottom: 20px;  
}
```

```
.stat-card {  
    background: var(--color-surface);  
    border: 1px solid var(--color-border);  
    border-radius: var(--radius);  
    box-shadow: var(--shadow);  
}
```



```
padding: 16px 20px;
display: flex;
flex-direction: column;
gap: 4px;
}
```

```
.stat-label {
  font-size: 0.75rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--color-muted);
}
```

```
.stat-value {
  font-size: 1.75rem;
  font-weight: 700;
  color: var(--color-primary);
  line-height: 1.2;
}
```

```
.table-panel {
  padding-bottom: 12px;
}
```

```
/* Table Toolbar */
.table-toolbar {
  display: flex;
  justify-content: space-between;
```

```
align-items: center;
margin-bottom: 12px;
padding: 8px 4px;
flex-wrap: wrap;
gap: 10px;
}
```

```
.table-toolbar-left,
.table-toolbar-right {
  display: flex;
  align-items: center;
  gap: 10px;
}
```

```
.table-toolbar-left input[type="checkbox"] {
  width: 18px;
  height: 18px;
  cursor: pointer;
}
```

```
.table-toolbar-left label {
  font-size: 0.9rem;
  font-weight: 500;
  color: var(--color-text);
  cursor: pointer;
}
```

```
.btn-delete-selected {
  background: var(--color-danger);
}
```

```
color: #fff;
padding: 8px 18px;
border: none;
border-radius: 8px;
font-weight: 600;
cursor: pointer;
transition: background 0.2s, transform 0.15s;
}
```

```
.btn-delete-selected:hover {
  background: var(--color-danger-hover);
}
```

```
.btn-delete-selected:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}
```

```
.table-wrapper {
  overflow-x: auto;
  border: 1px solid var(--color-border);
  border-radius: 8px;
}
```

```
table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.9rem;
}
```

```
th,  
td {  
    padding: 12px 14px;  
    text-align: left;  
    border-bottom: 1px solid var(--color-border);  
}
```

```
th {  
    background: #f8fafc;  
    font-size: 0.75rem;  
    font-weight: 700;  
    text-transform: uppercase;  
    letter-spacing: 0.04em;  
    color: var(--color-muted);  
    white-space: nowrap;  
    position: sticky;  
    top: 0;  
    z-index: 1;  
}
```

```
th input[type="checkbox"] {  
    width: 16px;  
    height: 16px;  
    cursor: pointer;  
}
```

```
td input[type="checkbox"] {  
    width: 16px;
```

```
height: 16px;  
cursor: pointer;  
}
```

```
tbody tr:last-child td {  
    border-bottom: none;  
}
```

```
tbody tr:hover {  
    background: #f8fafc;  
}
```

```
tbody tr.selected {  
    background: #dbeafe;  
}
```

```
.empty-row td {  
    text-align: center;  
    padding: 32px 16px;  
    color: var(--color-muted);  
    font-style: italic;  
}
```

```
.actions-cell {  
    white-space: nowrap;  
}
```

```
.btn-edit,  
.btn-delete {
```

```
padding: 6px 12px;  
font-size: 0.8rem;  
margin-right: 6px;  
}
```

```
.btn-edit {  
  background: var(--color-warning);  
  color: #fff;  
}
```

```
.btn-edit:hover {  
  background: #c2410c;  
}
```

```
.btn-delete {  
  background: var(--color-danger);  
  color: #fff;  
  margin-right: 0;  
}
```

```
.btn-delete:hover {  
  background: var(--color-danger-hover);  
}
```

```
.toast {  
  position: fixed;  
  bottom: 24px;  
  right: 24px;  
  max-width: 320px;
```

```
padding: 14px 18px;
border-radius: 8px;
font-weight: 500;
font-size: 0.9rem;
color: #fff;
box-shadow: 0 8px 24px rgba(0, 0, 0, 0.18);
z-index: 1000;
animation: slideIn 0.3s ease;
}
```

```
.toast-success {
  background: var(--color-success);
}
```

```
.toast-error {
  background: var(--color-danger);
}
```

```
.toast-info {
  background: #475569;
}
```

```
.hidden {
  display: none !important;
}
```

```
.visually-hidden {
  position: absolute;
  width: 1px;
```

```
height: 1px;
padding: 0;
margin: -1px;
overflow: hidden;
clip: rect(0, 0, 0, 0);
white-space: nowrap;
border: 0;
}
@keyframes slideIn {
  from {
    opacity: 0;
    transform: translateY(12px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
@media (max-width: 640px) {
  body {
    padding: 16px 12px 40px;
  }
  .panel {
    padding: 16px;
  }

  .toolbar {
    flex-direction: column;
    gap: 14px;
  }
}
```



```
}
```

```
.toolbar-group {  
  width: 100%;  
}
```

```
.btn-filter,  
.btn-sort {  
  flex: 1 1 auto;  
}
```

```
.toast {  
  left: 12px;  
  right: 12px;  
  bottom: 16px;  
  max-width: none;  
}
```

```
th,  
td {  
  padding: 10px;  
  font-size: 0.82rem;  
}
```

```
.table-toolbar {  
  flex-direction: column;  
  align-items: flex-start;  
}  
}
```

Script.js

```
"use strict";
```

```
const STORAGE_KEY = "students";
```

```
let students = loadStudents();
```

```
let activeFilter = "all";
```

```
let activeSort = null;
```

```
const els = {
```

```
  form: document.getElementById("studentForm"),
```

```
  editId: document.getElementById("editId"),
```

```
  studentId: document.getElementById("studentId"),
```

```
  name: document.getElementById("name"),
```

```
  email: document.getElementById("email"),
```

```
  mobile: document.getElementById("mobile"),
```

```
  course: document.getElementById("course"),
```

```
  age: document.getElementById("age"),
```

```
  percentage: document.getElementById("percentage"),
```

```
  submitBtn: document.getElementById("submitBtn"),
```

```
  cancelBtn: document.getElementById("cancelBtn"),
```

```
  deleteLastBtn: document.getElementById("deleteLastBtn"),
```

```
  search: document.getElementById("search"),
```

```
  tableBody: document.getElementById("studentTableBody"),
```

```
  totalStudents: document.getElementById("totalStudents"),
```

```
  avgPercentage: document.getElementById("avgPercentage"),
```

```
  showingCount: document.getElementById("showingCount"),
```

```
  toast: document.getElementById("toast"),
```

```
selectedCount: document.getElementById("selectedCount"),
deleteSelectedBtn: document.getElementById("deleteSelectedBtn"),
selectAll: document.getElementById("selectAll"),
selectAllHeader: document.getElementById("selectAllHeader"),
};
```

```
function loadStudents() {
  try {
    const data = JSON.parse(localStorage.getItem(STORAGE_KEY));
    return Array.isArray(data) ? data : [];
  } catch {
    return [];
  }
}
```

```
function saveStudents() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(students));
}
```

```
function escapeHtml(text) {
  const div = document.createElement("div");
  div.textContent = String(text);
  return div.innerHTML;
}
```

```
function showToast(message, type = "success") {
  els.toast.textContent = message;
  els.toast.className = `toast toast-${type}`;
  clearTimeout(showToast._timer);
```

```
showToast._timer = setTimeout(() => {  
    els.toast.classList.add("hidden");  
}, 3000);  
}  
  
function getStudentById(id) {  
    return students.find((s) => s.studentId === id);  
}  
  
function isDuplicateId(id, excludeId = "") {  
    return students.some((s) => s.studentId === id && s.studentId !== excludeId);  
}  
  
function validateForm() {  
    const id = els.studentId.value.trim();  
    const editingId = els.editId.value;  
  
    if (!id) {  
        showToast("Student ID is required.", "error");  
        els.studentId.focus();  
        return false;  
    }  
  
    if (isDuplicateId(id, editingId)) {  
        showToast("A student with this ID already exists.", "error");  
        els.studentId.focus();  
        return false;  
    }  
}
```

```
const age = Number(els.age.value);
const pct = Number(els.percentage.value);

if (age < 5 || age > 100) {
    showToast("Age must be between 5 and 100.", "error");
    els.age.focus();
    return false;
}

if (pct < 0 || pct > 100) {
    showToast("Percentage must be between 0 and 100.", "error");
    els.percentage.focus();
    return false;
}

return true;
}

function buildStudentFromForm() {
    return {
        studentId: els.studentId.value.trim(),
        name: els.name.value.trim(),
        email: els.email.value.trim(),
        mobile: els.mobile.value.trim(),
        course: els.course.value.trim(),
        age: Number(els.age.value),
        percentage: Number(els.percentage.value),
    };
}
```

```
function resetForm() {  
    els.form.reset();  
    els.editId.value = "";  
    els.studentId.disabled = false;  
    els.submitBtn.textContent = "💾 Save Student";  
    els.cancelBtn.classList.add("hidden");  
}
```

```
function setEditMode(student) {  
    els.editId.value = student.studentId;  
    els.studentId.value = student.studentId;  
    els.studentId.disabled = true;  
    els.name.value = student.name;  
    els.email.value = student.email;  
    els.mobile.value = student.mobile;  
    els.course.value = student.course;  
    els.age.value = student.age;  
    els.percentage.value = student.percentage;  
    els.submitBtn.textContent = "✎ Update Student";  
    els.cancelBtn.classList.remove("hidden");  
    els.studentId.scrollIntoView({ behavior: "smooth", block: "center" });  
}
```

```
function applyFilter(list) {  
    switch (activeFilter) {  
        case "percentage":  
            return list.filter((s) => s.percentage > 75);  
    }
```

```
case "age":
    return list.filter((s) => s.age > 20);
default:
    return list;
}
}

function applySearch(list) {
    const keyword = els.search.value.trim().toLowerCase();
    if (!keyword) return list;

    return list.filter(
        (s) =>
            s.name.toLowerCase().includes(keyword) ||
            s.course.toLowerCase().includes(keyword) ||
            s.studentId.toLowerCase().includes(keyword)
    );
}

function applySort(list) {
    const sorted = [...list];

    switch (activeSort) {
        case "name":
            sorted.sort((a, b) => a.name.localeCompare(b.name));
            break;
        case "age":
            sorted.sort((a, b) => a.age - b.age);
            break;
    }
}
```

```
case "percentage":
    sorted.sort((a, b) => b.percentage - a.percentage);
    break;
}

return sorted;
}

function getVisibleStudents() {
    return applySort(applySearch(applyFilter(students)));
}

function updateStats(visibleCount) {
    els.totalStudents.textContent = students.length;
    els.showingCount.textContent = visibleCount;

    if (students.length === 0) {
        els.avgPercentage.textContent = "0";
        return;
    }

    const avg =
        students.reduce((sum, s) => sum + s.percentage, 0) / students.length;
    els.avgPercentage.textContent = avg.toFixed(2);
}

function updateSelectedCount() {
    const checkboxes = document.querySelectorAll('#studentTableBody
input[type="checkbox"]');
```



```
const checked = document.querySelectorAll('#studentTableBody
input[type="checkbox"]:checked');

els.selectedCount.textContent = checked.length;

els.deleteSelectedBtn.disabled = checked.length === 0;
}
```

```
function renderTable(data) {
  els.tableBody.replaceChildren();

  if (data.length === 0) {
    const row = document.createElement("tr");
    row.className = "empty-row";
    row.innerHTML =
      '<td colspan="9">No students found. Add a student or adjust your search.</td>';
    els.tableBody.appendChild(row);
    updateStats(0);
    updateSelectedCount();
    return;
  }
```

```
const fragment = document.createDocumentFragment();
```

```
data.forEach((student, index) => {
  const row = document.createElement("tr");
  row.dataset.index = index;
  row.innerHTML = `
    <td>
      <input type="checkbox" class="student-checkbox" data-
id="${escapeHtml(student.studentId)}" onchange="updateSelectedCount()">
```

```

</td>
<td>${escapeHtml(student.studentId)}</td>
<td>${escapeHtml(student.name)}</td>
<td>${escapeHtml(student.email)}</td>
<td>${escapeHtml(student.mobile)}</td>
<td>${escapeHtml(student.course)}</td>
<td>${escapeHtml(student.age)}</td>
<td>${escapeHtml(student.percentage)}%</td>
<td class="actions-cell">
    <button type="button" class="btn btn-edit" data-action="edit" data-
id="${escapeHtml(student.studentId)}">Edit</button>
    <button type="button" class="btn btn-delete" data-action="delete" data-
id="${escapeHtml(student.studentId)}">Delete</button>
</td>
`;
fragment.appendChild(row);
});

els.tableBody.appendChild(fragment);
updateStats(data.length);
updateSelectedCount();
}

function displayStudents() {
    renderTable(getVisibleStudents());
}

function handleSubmit(e) {
    e.preventDefault();

```

```
if (!validateForm()) return;

const student = buildStudentFromForm();
const editingId = els.editId.value;

if (editingId) {
  const index = students.findIndex((s) => s.studentId === editingId);
  if (index !== -1) {
    students[index] = student;
    showToast("Student updated successfully.");
  }
} else {
  students.push(student);
  showToast("Student added successfully.");
}

saveStudents();
resetForm();
displayStudents();
}

function handleEdit(id) {
  const student = getStudentById(id);
  if (!student) {
    showToast("Student not found.", "error");
    displayStudents();
    return;
  }
}
```

```
setEditMode(student);
}

function handleDelete(id) {
  const student = getStudentById(id);
  if (!student) {
    showToast("Student not found.", "error");
    displayStudents();
    return;
  }

  const confirmed = confirm(`Delete ${student.name} (${student.studentId})?`);
  if (!confirmed) return;

  students = students.filter((s) => s.studentId !== id);
  saveStudents();

  if (els.editId.value === id) resetForm();

  showToast(`Student ${student.name} deleted.`, "info");
  displayStudents();
}

// Delete only the last student in the list (bottom to top)
function deleteLastStudent() {
  if (students.length === 0) {
    showToast("No students to delete.", "error");
    return;
  }
}
```

```
// Get the last student
const lastStudent = students[students.length - 1];

const confirmed = confirm(`Delete the last student?\n\n${lastStudent.name}
(${lastStudent.studentId}) will be deleted.`);
if (!confirmed) return;

// Remove the last student
students.pop();
saveStudents();

if (els.editId.value === lastStudent.studentId) {
    resetForm();
}

showToast(`Last student ${lastStudent.name} deleted.`, "info");
displayStudents();
}

// Delete only selected students
function deleteSelected() {
    const checkboxes = document.querySelectorAll('#studentTableBody
input[type="checkbox"]:checked');
    if (checkboxes.length === 0) {
        showToast("No students selected.", "error");
        return;
    }
}
```

```

const ids = Array.from(checkboxes).map(cb => cb.dataset.id);
const names = ids.map(id => {
  const student = getStudentById(id);
  return student ? student.name : id;
});

const confirmed = confirm(`Delete ${ids.length} selected student(s)?\n\nOnly the
checked students will be deleted.`);
if (!confirmed) return;

// Delete only the selected students (filter them out)
const deletedCount = ids.length;
students = students.filter((s) => !ids.includes(s.studentId));
saveStudents();

if (els.editId.value && ids.includes(els.editId.value)) {
  resetForm();
}

// Uncheck select all checkboxes
if (els.selectAll) els.selectAll.checked = false;
if (els.selectAllHeader) els.selectAllHeader.checked = false;

displayStudents();
showToast(`${deletedCount} student(s) deleted successfully`, "success");
}

// Toggle all checkboxes
function toggleAllCheckboxes() {

```

```
const checkboxes = document.querySelectorAll('#studentTableBody
input[type="checkbox"]');

const isChecked = els.selectAll.checked || els.selectAllHeader.checked;

checkboxes.forEach(cb => {
    cb.checked = isChecked;
});

// Sync both select all checkboxes
if (els.selectAll) els.selectAll.checked = isChecked;
if (els.selectAllHeader) els.selectAllHeader.checked = isChecked;

updateSelectedCount();
}

// Make functions globally accessible
window.toggleAllCheckboxes = toggleAllCheckboxes;
window.deleteSelected = deleteSelected;
window.updateSelectedCount = updateSelectedCount;

function setActiveFilter(filter) {
    activeFilter = filter;
    document.querySelectorAll("[data-filter]").forEach((btn) => {
        btn.classList.toggle("active", btn.dataset.filter === filter);
    });
    displayStudents();
}

function setActiveSort(sort) {
```

```
activeSort = activeSort === sort ? null : sort;

document.querySelectorAll("[data-sort]").forEach((btn) => {
    btn.classList.toggle("active", btn.dataset.sort === activeSort);
});

displayStudents();
}

els.form.addEventListener("submit", handleSubmit);

els.cancelBtn.addEventListener("click", resetForm);

// Delete Last Student button
if (els.deleteLastBtn) {
    els.deleteLastBtn.addEventListener("click", deleteLastStudent);
}

els.search.addEventListener("input", displayStudents);

document.querySelectorAll("[data-filter]").forEach((btn) => {
    btn.addEventListener("click", () => setActiveFilter(btn.dataset.filter));
});

document.querySelectorAll("[data-sort]").forEach((btn) => {
    btn.addEventListener("click", () => setActiveSort(btn.dataset.sort));
});

// Sync select all checkboxes
if (els.selectAll) {
    els.selectAll.addEventListener('change', toggleAllCheckboxes);
```



```

    }

    if (els.selectAllHeader) {
        els.selectAllHeader.addEventListener('change', toggleAllCheckboxes);
    }

    els.tableBody.addEventListener("click", (e) => {
        const btn = e.target.closest("[data-action]");
        if (!btn) return;

        const id = btn.dataset.id;
        if (btn.dataset.action === "edit") handleEdit(id);
        if (btn.dataset.action === "delete") handleDelete(id);
    });

    // Initial display
    displayStudents();

```

OUTPUT:-

Student Management System
Add, search, filter, and manage student records

Student Details

STUDENT ID	NAME	EMAIL	MOBILE	COURSE
e.g. STU001	Full name	email@example.com	10-digit number	e.g. Computer Science
AGE	PERCENTAGE			
0-100	0-100			

[Save Student](#)
[Delete Last](#)

Search & Filters

Search by name, ID, or course...

FILTER
[Percentage > 75](#)
[Age > 20](#)
[Show All](#)

Sort
[Name](#)
[Age](#)
[Percentage](#)

TOTAL STUDENTS
3

AVERAGE PERCENTAGE
90.33

SHOWING
3

Student List

☐ Select All
[Delete Selected \(0\)](#)

☐	ID	NAME	EMAIL	MOBILE	COURSE	AGE	PERCENTAGE	ACTIONS
☐	45	Devan	23ss02t187@ppsu.ac.in	+917861976128	BSCIT	22	89%	Edit Delete
☐	56	Rishabh Shah	23ss02t181@ppsu.ac.in	7861976128	BSCIT	22	90%	Edit Delete
☐	57	Tanisha	23ss02mb061@ppsu.ac.in	7861976128	BSCIT	21	92%	Edit Delete